Flight Delay Big Data Analysis - Final Report

Luca Foresti

2026-05-21

Contents

1 Executive Summary

This project analyzes the 2024 United States flight delay dataset with a reproducible big-data workflow. The repository prepares a canonical cleaned Parquet dataset, implements two analytical jobs with three big-data technologies, validates the outputs, records benchmark evidence, and produces the tables and figures included in this PDF.

Repository: https://github.com/Forest904/flight-delay-big-data-analysis.git

The implemented analyses are:

- **Analysis 1:** delay report by departure airport, month, and departure-delay range.
- **Analysis 2:** airline-airport ranking by average departure delay.

The three selected technologies are Spark SQL, Spark Core RDDs, and Hive. Spark SQL is used as the correctness reference because its declarative aggregations and window functions make the required output rules easiest to review. Spark Core reimplements the same logic with lower-level RDD transformations. Hive provides the required SQL-on-Hadoop comparison point. MapReduce is intentionally out of scope for this submission.

The main reproducibility commands are:

```
make setup
make check-env
make prepare
make generate-sizes
make run-spark-sql
make run-spark-core
make run-hive
make benchmark-local
make benchmark-cluster
make charts
make report
```

The `make benchmark-cluster` target name is kept for repository compatibility, but in this project it runs Docker standalone simulation evidence, not a true multi-machine cluster.

2 Assignment Coverage

Final-report requirement	Where it is answered
Data preparation operations	Dataset And Preparation
Implementation choices and pseudocode	Analyses And Implementations
First 10 result rows	Evidence Appendix
Execution-time tables and charts	Benchmark Evidence
Local and cluster-style execution settings	Docker Standalone Simulation and Benchmark Evidence

Final-report requirement	Where it is answered
Critical discussion	Critical Discussion
GitHub repository link	Executive Summary

3 Dataset And Preparation

The source dataset is Kaggle `hrishitpatil/flight-data-2024`, stored locally as `data/raw/flight_data_2024.csv`. The raw CSV has 7,079,081 rows, 35 columns, and a local file size of 1,309,010,752 bytes. The raw file is not committed to Git because it is large and externally hosted.

The preparation pipeline is implemented by `src/preparation/prepare_spark.py` and is run with:

```
make prepare
```

The prepared output is `data/prepared/flights_2024_clean.parquet`. This Parquet dataset is the canonical input for Spark SQL, Spark Core, and Hive. Using one prepared source for all technologies keeps the comparison fair: each implementation receives the same normalized schema, the same row-retention policy, and the same benchmark input files.

3.1 Canonical Fields

The prepared dataset keeps only the fields needed by the selected analyses:

- Date and grouping: `flight_date`, `month`.
- Airline fields: `airline_code`, plus nullable `airline_name` because the raw dataset has no stable airline-name field.
- Airport fields: `origin_airport`, `destination_airport`.
- Delay metrics: `departure_delay`, `arrival_delay`.
- Cancellation and diversion: `cancelled`, `diverted`, `cancellation_code`.
- Delay causes: carrier, weather, NAS, security, and late-aircraft delay minutes.

3.2 Cleaning Policy

The cleaning policy is conservative. Rows are removed only when structural fields are invalid after casting: missing or invalid flight date, month, airline code, origin airport, destination airport, cancelled flag, or diverted flag. Cancelled flights, diverted flights, null delay values, and negative delay values are preserved.

Empty strings are normalized to null. Numeric delay fields are cast to numeric types but are not imputed. This matters because Spark and Hive averages ignore null values, while cancellation-rate calculations still need cancelled flights in the denominator.

The raw data inspection found 92,970 null departure-delay rows and 113,814 null arrival-delay rows. These rows are retained unless they fail a structural rule. Negative delay values are preserved because they represent early departures or arrivals, not data errors.

3.3 Generated Input Sizes

The input-size generator creates controlled benchmark inputs under `data/generated/`:

- `100k`
- `500k`
- `1m`
- `3m`
- `full`

Smaller inputs are selected with a deterministic hash-based method using seed `20240520`. This avoids chronological bias from simply taking the first rows of the CSV. Optional larger inputs, such as `14m` and `28m`, can be generated by controlled replication when explicitly requested, but they were not required for the current report artifacts.

4 Analyses And Implementations

4.1 Analysis 1: Delay By Airport, Month, And Delay Range

This job satisfies the assignment requirement to report delay behavior for each departure airport and month. It groups rows by:

- `origin_airport`
- `month`
- `delay_range`

The delay ranges are:

- `low`: `departure_delay < 15`
- `medium`: `15 <= departure_delay <= 60`
- `high`: `departure_delay > 60`

Rows with null departure delay are excluded from this job because they cannot be assigned to one of the required ranges. For each group, the output reports flight count, average departure delay, average arrival delay, and the three most frequent delay or cancellation causes.

The stable output schema includes:

Field group	Columns
Grouping keys	`origin_airport`, `month`, `delay_range`
Metrics	`flight_count`, `avg_departure_delay`, `avg_arrival_delay`
Top causes	`top_1_cause`, `top_1_count`, `top_2_cause`, `top_2_count`, `top_3_cause`, `top_3_count`

Cause labels are derived from cancellation codes or the largest positive delay-cause field. Cause ties are deterministic: cause count descending, then cause label ascending. Groups with fewer than three available causes use an empty cause value and count `0`.

Textual pseudocode:

```
filter rows where departure_delay is not null
derive delay_range from departure_delay
derive cause from cancellation code or largest positive delay-cause field
group by origin_airport, month, delay_range
compute flight count and average delays
count causes inside each group
rank causes by count descending and cause label ascending
emit the first three causes and pad missing slots with count 0
order output deterministically
```

4.2 Analysis 2: Airline-Airport Ranking

This job satisfies the assignment requirement to compare each airline at each departure airport against the airport average. For each `(origin_airport, airline_code)` pair it computes:

- flight count;
- average departure delay;

- average arrival delay;
- cancellation rate;
- airport average departure delay;
- difference from the airport average;
- rank at the airport, from lowest average departure delay to highest.

Textual pseudocode:

```
group flights by origin_airport and airline_code
compute airline-level counts, averages, and cancellation rate
group flights by origin_airport
compute airport-level average departure delay
join airline metrics with airport metrics
rank airlines per airport by avg_departure_delay ascending, nulls last
order output by airport, rank, average delay, and airline code
```

4.3 Spark SQL

Spark SQL is the reference implementation. It expresses both analyses with DataFrame temporary views, SQL aggregations, and window functions. The ranking job uses `RANK() OVER (PARTITION BY origin_airport ORDER BY avg_departure_delay ASC NULLS LAST)`. The delay job uses grouped cause counts and `ROW_NUMBER()` to select the top three causes with deterministic tie-breaking.

Spark SQL is the most concise implementation in this project. The grouping keys, derived fields, ranking rule, and top-three cause rule are visible in the query structure. The cost is that grouped aggregations and window functions still require shuffles; declarative syntax makes the logic easier to maintain, but it does not remove distributed execution cost.

4.4 Spark Core

Spark Core reimplements the Spark SQL logic with RDD transformations. It uses `reduceByKey` for delay aggregates, cause counts, airline aggregates, and airport aggregates. This avoids `groupByKey` for large aggregations and allows partial accumulator state to be combined before the shuffle.

The Spark Core implementation is intentionally explicit. It builds accumulator objects for counts, sums, cancellation totals, and cause counts; joins compact aggregate RDDs by airport; and sorts per-airport airline rows to reproduce the Spark SQL rank semantics. This gives more control over the physical steps but also makes the implementation more verbose and easier to get wrong.

4.5 Hive

Hive is the third required technology. The runner starts the Docker Compose Hive stack, creates an external table over the prepared Parquet data, executes HiveQL versions of both analyses, and exports CSV outputs with the same schemas as Spark SQL and Spark Core.

Hive is useful as a SQL-on-Hadoop comparison point. Its SQL syntax is familiar for aggregation-heavy work, but this project runs Hive as a local containerized HiveServer2/metastore/PostgreSQL stack. The setup does not use a distributed Hive-on-YARN or HDFS deployment, so the Hive results should be read as controlled local/container evidence rather than as production distributed Hive performance.

5 Docker Standalone Simulation

The Docker standalone simulation is the repository's controlled alternative execution setting. Spark uses Docker Compose with one Spark standalone master, two Spark workers, and a driver service. The workers are separate containers and processes, but they all run on one physical laptop and share Docker Desktop

CPU, memory, and disk limits. Data is read from a bind-mounted project directory, not HDFS or object storage.

Hive is included in the Docker benchmark CSV so the report contains rows for all three selected technologies, but Hive remains a single-node containerized Hive setup. Therefore the report uses the phrase "Docker standalone simulation" and avoids presenting these runs as real multi-machine cluster performance.

The documented topology is stored in `docs/cluster_simulation.md`. The command used to run the simulation evidence is:

```
make benchmark-cluster
```

6 Environment And Runtime Configuration

The environment summary was generated by `scripts/generate_environment_summary.py`. The complete artifact is stored at `report/tables/environment_summary.md`.

Category	Item	Value
Host OS	Windows detail	Microsoft Windows 11 Home 10.0.26200 build 26200 64-bit
CPU	Model	AMD Ryzen AI 7 350 w/ Radeon 860M
CPU	Cores	8 physical, 16 logical
Memory	Host RAM	33,598,853,120 bytes
Disk	Project drive	NVMe SSD, 1,024,209,543,168 bytes
Python	Version	3.12.2
Java	Version	OpenJDK 17.0.19
PySpark	Version	4.1.1
Hive	Base image	`apache/hive:4.0.1`
Docker	Version	29.4.3
Docker Compose	Version	v5.1.3
Docker Desktop	Limits	16 CPUs, 16,391,536,640 memory bytes
Local Spark	Master and partitions	`local[*]`, 8 shuffle partitions
Docker Spark	Master and partitions	`spark://spark-master:7077`, 16 shuffle partitions
Docker topology	Spark	1 master, 2 workers, 2 cores and 2 GB memory per worker
Docker topology	Hive	HiveServer2, metastore, PostgreSQL, single host

7 Benchmark Evidence

The benchmark runner records technology, job name, input size, environment, execution-setting label, duration, output rows, status, timestamp, input path, and metrics path. The report-ready artifacts are generated under `report/tables/` and `report/figures/`.

The latest M2 benchmark campaign completed all expected rows:

Environment	Inputs	Technologies	Jobs	Status
Local	`100k`, `500k`, `1m`, `3m`, `full`	Spark SQL, Spark Core, Hive	both jobs	30/30 successful
Docker standalone simulation	`100k`, `500k`, `1m`	Spark SQL, Spark Core, Hive	both jobs	18/18 successful

The local run ID is `20260520T161558752230Z`. The Docker standalone simulation run ID is `20260520T162530812695Z`. The full status matrix is stored in `report/tables/benchmark_status.md`.

7.1 Benchmark Pivot

environment	input_label	records	job_name	Spark SQL s	Spark Core s	Hive s
docker-simulation	100k	100000	airline_airport_ranking	5.136	2.148	9.189
docker-simulation	100k	100000	delay_by_airport_month	9.642	2.386	12.267
docker-simulation	500k	500000	airline_airport_ranking	5.164	2.191	8.986
docker-simulation	500k	500000	delay_by_airport_month	9.393	2.505	12.046
docker-simulation	1m	1000000	airline_airport_ranking	3.902	2.112	8.889
docker-simulation	1m	1000000	delay_by_airport_month	8.765	2.638	13.03
local	100k	100000	airline_airport_ranking	1.291	0.552	8.993
local	100k	100000	delay_by_airport_month	6.666	0.713	12.075
local	500k	500000	airline_airport_ranking	1.491	0.567	9.102
local	500k	500000	delay_by_airport_month	7.164	0.763	12.926
local	1m	1000000	airline_airport_ranking	1.693	0.624	8.972
local	1m	1000000	delay_by_airport_month	8.388	0.857	13.009
local	3m	3000000	airline_airport_ranking	2.201	0.607	12.141
local	3m	3000000	delay_by_airport_month	9.2	0.957	14.728
local	full	7079081	airline_airport_ranking	2.204	0.592	12.699
local	full	7079081	delay_by_airport_month	9.066	0.898	19.878

7.2 Rows Per Second

Rows-per-second values are computed as input records divided by elapsed seconds. They show that fixed startup and planning costs dominate small inputs: throughput often increases sharply even when elapsed seconds change only slightly.

environment	input	job	Spark SQL rows/s	Spark Core rows/s	Hive rows/s
docker-simulation	100k	airline_airport_ranking	19470.807	46560.874	10882.366
docker-simulation	100k	delay_by_airport_month	10371.378	41911.588	8151.962
docker-simulation	500k	airline_airport_ranking	96828.274	228218.694	55644.414
docker-simulation	500k	delay_by_airport_month	53233.26	199606.615	41505.914
docker-simulation	1m	airline_airport_ranking	256255.583	473416.033	112496.493
docker-simulation	1m	delay_by_airport_month	114084.925	379025.775	76747.944
local	100k	airline_airport_ranking	77484.482	181312.485	11120.294
local	100k	delay_by_airport_month	15001.178	140269.177	8281.492
local	500k	airline_airport_ranking	335425.944	882604.743	54930.737
local	500k	delay_by_airport_month	69797.25	655615.61	38681.203
local	1m	airline_airport_ranking	590656.29	1601365.645	111458.577
local	1m	delay_by_airport_month	119217.22	1166350.78	76868.967
local	3m	airline_airport_ranking	1363059.542	4945500.584	247099.83
local	3m	delay_by_airport_month	326079.726	3136218.515	203697.199
local	full	airline_airport_ranking	3211817.848	11948864.71	557468.879
local	full	delay_by_airport_month	780827.038	7883067.134	356119.936

7.3 Speedup Ratios

Each value is a duration ratio. A value above `1` means the numerator took longer than the denominator in that run.

environment	input	job	SQL/Core	Hive/SQL	Hive/Core
docker-simulation	100k	airline_airport_ranking	2.391	1.789	4.279

environment	input	job	SQL/Core	Hive/SQL	Hive/Core
docker-simulation	100k	delay_by_airport_month	4.041	1.272	5.141
docker-simulation	500k	airline_airport_ranking	2.357	1.74	4.101
docker-simulation	500k	delay_by_airport_month	3.75	1.283	4.809
docker-simulation	1m	airline_airport_ranking	1.847	2.278	4.208
docker-simulation	1m	delay_by_airport_month	3.322	1.486	4.939
local	100k	airline_airport_ranking	2.34	6.968	16.305
local	100k	delay_by_airport_month	9.351	1.811	16.938
local	500k	airline_airport_ranking	2.631	6.106	16.068
local	500k	delay_by_airport_month	9.393	1.804	16.949
local	1m	airline_airport_ranking	2.711	5.299	14.367
local	1m	delay_by_airport_month	9.783	1.551	15.173
local	3m	airline_airport_ranking	3.628	5.516	20.014
local	3m	delay_by_airport_month	9.618	1.601	15.396
local	full	airline_airport_ranking	3.72	5.761	21.434
local	full	delay_by_airport_month	10.096	2.193	22.136

7.4 Normalized Scalability

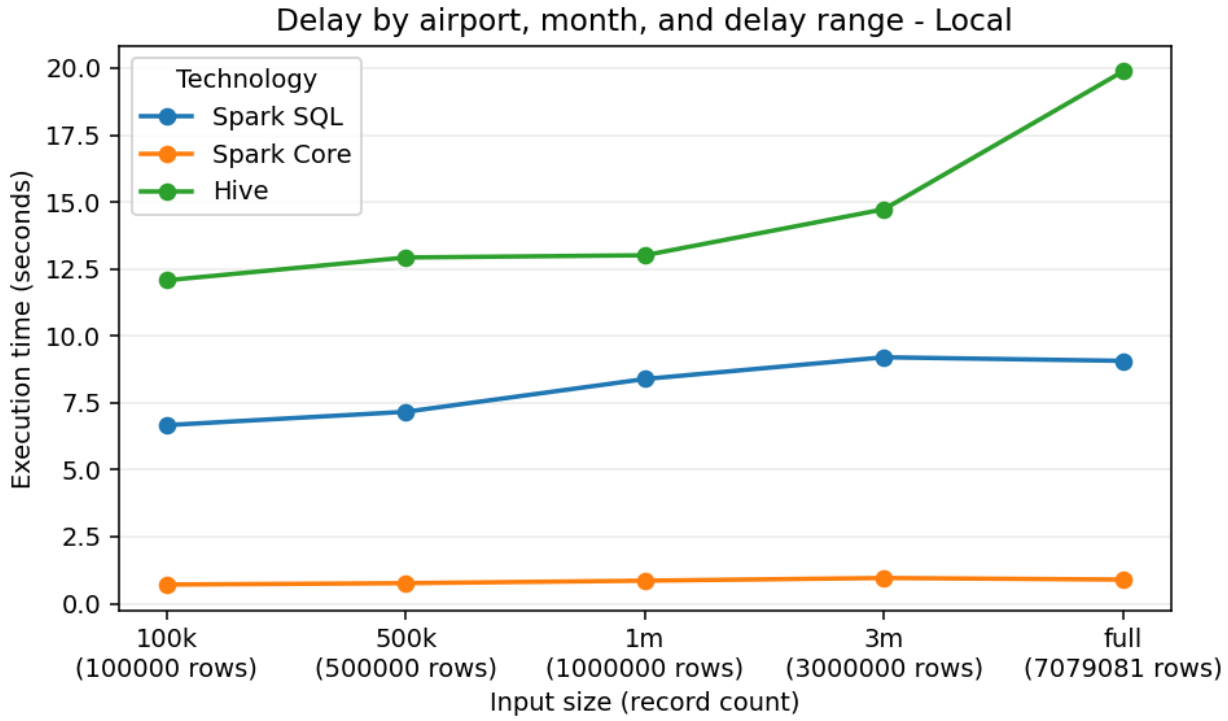
The following table normalizes duration, record count, and throughput against the `100k` baseline for each environment, job, and technology. For example, a `records_vs_100k` value of `10` means the input has ten times as many records as the baseline. The compact columns `dur`, `rec`, and `thr` mean `duration_vs_100k`, `records_vs_100k`, and `throughput_vs_100k`.

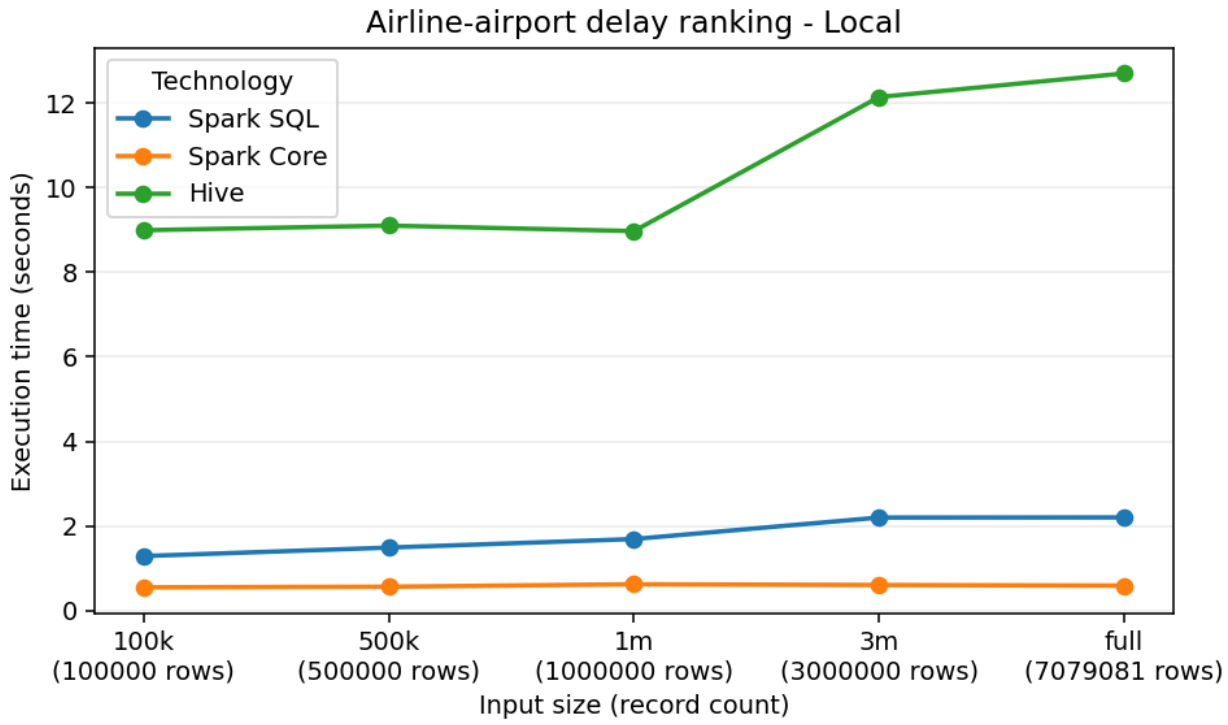
env	input	job	tech	dur	rec	thr
docker	100k	ranking	Hive	1	1	1
docker	500k	ranking	Hive	0.978	5	5.113
docker	1m	ranking	Hive	0.967	10	10.338
docker	100k	ranking	Core	1	1	1
docker	500k	ranking	Core	1.02	5	4.902
docker	1m	ranking	Core	0.984	10	10.168
docker	100k	ranking	SQL	1	1	1
docker	500k	ranking	SQL	1.005	5	4.973
docker	1m	ranking	SQL	0.76	10	13.161
docker	100k	delay	Hive	1	1	1
docker	500k	delay	Hive	0.982	5	5.092
docker	1m	delay	Hive	1.062	10	9.415
docker	100k	delay	Core	1	1	1
docker	500k	delay	Core	1.05	5	4.763
docker	1m	delay	Core	1.106	10	9.043
docker	100k	delay	SQL	1	1	1
docker	500k	delay	SQL	0.974	5	5.133
docker	1m	delay	SQL	0.909	10	11
local	100k	ranking	Hive	1	1	1
local	500k	ranking	Hive	1.012	5	4.94
local	1m	ranking	Hive	0.998	10	10.023
local	3m	ranking	Hive	1.35	30	22.221
local	full	ranking	Hive	1.412	70.791	50.131
local	100k	ranking	Core	1	1	1
local	500k	ranking	Core	1.027	5	4.868
local	1m	ranking	Core	1.132	10	8.832
local	3m	ranking	Core	1.1	30	27.276

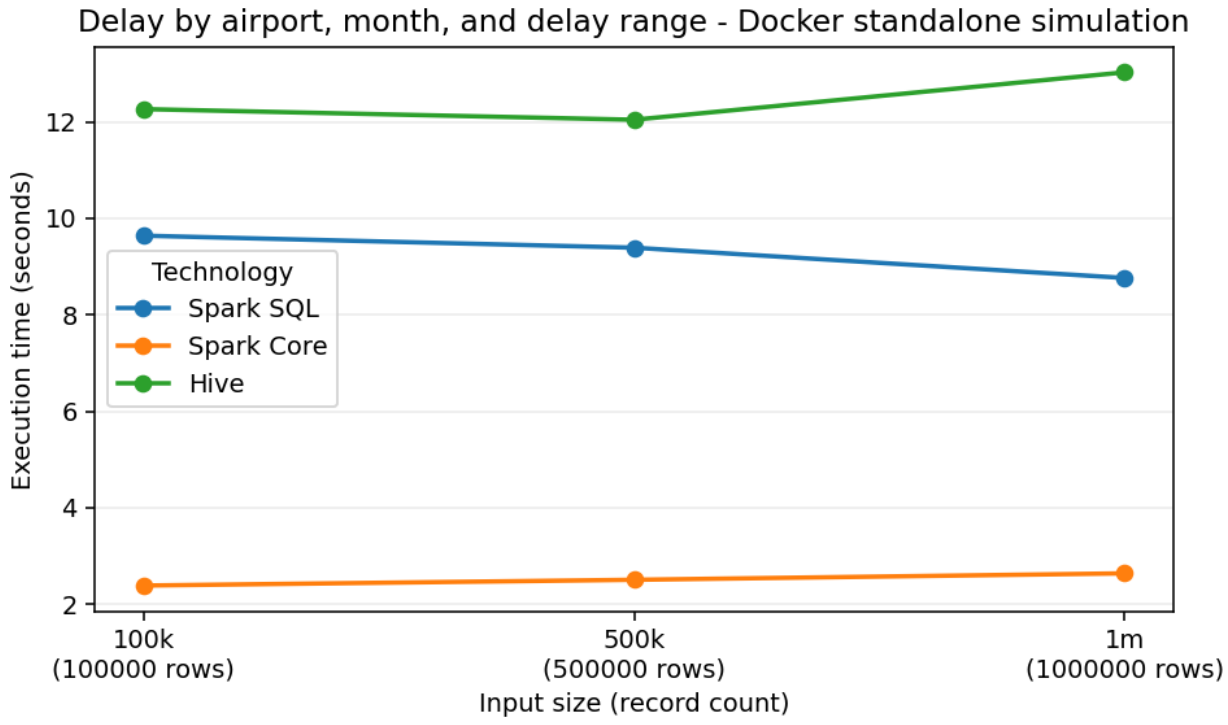
env	input	job	tech	dur	rec	thr
local	full	ranking	Core	1.074	70.791	65.902
local	100k	ranking	SQL	1	1	1
local	500k	ranking	SQL	1.155	5	4.329
local	1m	ranking	SQL	1.312	10	7.623
local	3m	ranking	SQL	1.705	30	17.591
local	full	ranking	SQL	1.708	70.791	41.451
local	100k	delay	Hive	1	1	1
local	500k	delay	Hive	1.07	5	4.671
local	1m	delay	Hive	1.077	10	9.282
local	3m	delay	Hive	1.22	30	24.597
local	full	delay	Hive	1.646	70.791	43.002
local	100k	delay	Core	1	1	1
local	500k	delay	Core	1.07	5	4.674
local	1m	delay	Core	1.203	10	8.315
local	3m	delay	Core	1.342	30	22.359
local	full	delay	Core	1.26	70.791	56.2
local	100k	delay	SQL	1	1	1
local	500k	delay	SQL	1.075	5	4.653
local	1m	delay	SQL	1.258	10	7.947
local	3m	delay	SQL	1.38	30	21.737
local	full	delay	SQL	1.36	70.791	52.051

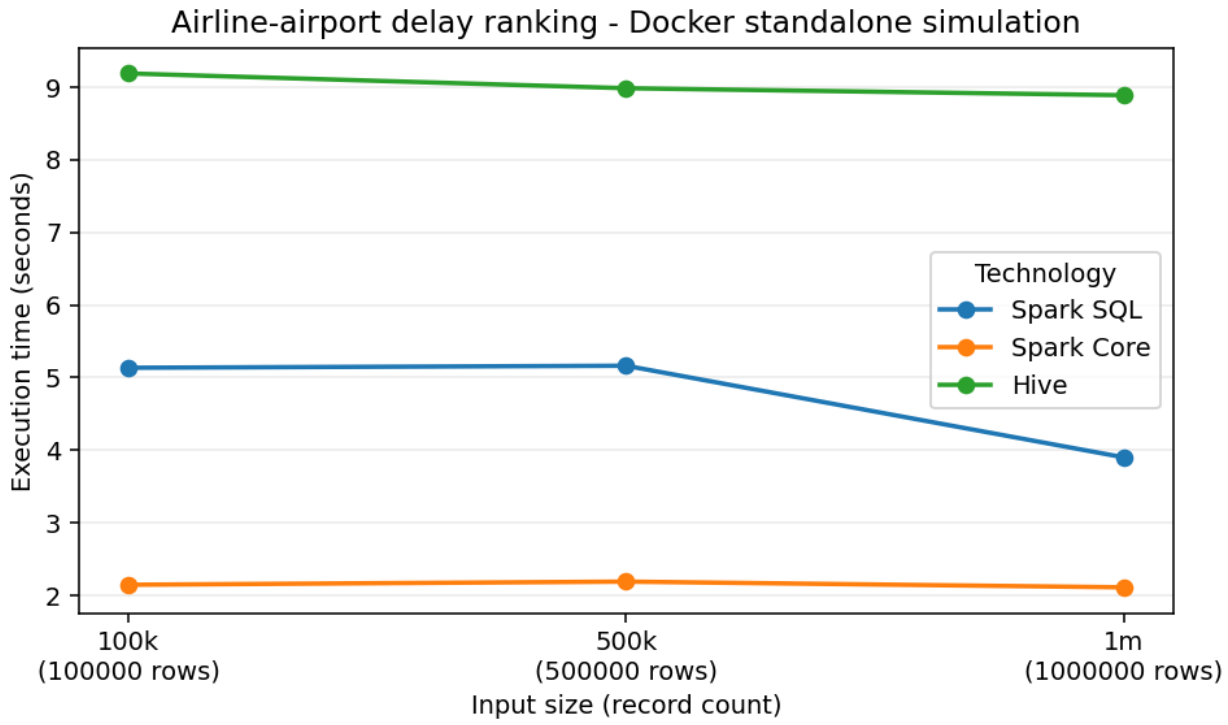
7.5 Benchmark Charts

Execution-time charts are generated separately for local execution and Docker standalone simulation. Charts use line plots only when at least three input sizes are available for a job and environment.




Airline-airport delay ranking - Local
Technology
Spark SQL
Spark Core
Hive
Execution time (seconds)
0
2
4
6
8
10
12
100k
(100000 rows)
500k
(500000 rows)
1m
(1000000 rows)
3m
(3000000 rows)
full
(7079081 rows)
Input size (record count)


Delay by airport, month, and delay range - Docker standalone simulation
Technology
Spark SQL
Spark Core
Hive
Execution time (seconds)
2
4
6
8
10
12
100k
(100000 rows)
500k
(500000 rows)
1m
(1000000 rows)
Input size (record count)



8 Critical Discussion

Spark SQL is the most expressive implementation. It is especially strong for the two selected analyses because both are naturally relational: they derive columns, group by keys, aggregate measures, and rank rows inside partitions. Window functions make the airline ranking and top-three cause selection clear. The tradeoff is that the physical plan still has to shuffle data for grouped aggregations and window partitions. Spark SQL hides some mechanics, but it does not make those costs disappear.

Spark Core is the most explicit implementation. The RDD version exposes every map, key construction, accumulator merge, join, and sort. Its accumulator design is useful because it combines counts and sums before the shuffle and keeps the top-three cause logic deterministic. That explicitness also makes the code more verbose. Reproducing SQL ranking semantics, null handling, and top-three padding requires careful manual logic that Spark SQL expresses more directly.

Hive provides SQL-on-Hadoop style expressiveness, but it has the highest overhead in this project. The containerized Hive stack pays service, query startup, metastore, and export costs that are visible in the benchmark rows. Hive remains useful as a third technology and as a contrast with Spark, but the current local container setup does not demonstrate the strengths of a large distributed Hive deployment.

The delay-by-airport-month job is generally more expensive than the airline-airport ranking. It derives delay ranges, maps cancellation or delay causes, counts causes inside each airport-month-range group, and ranks causes to emit three positions. These steps increase shuffle and aggregation pressure. The ranking job also performs grouped aggregation and per-airport ranking, but after the first aggregation it works with compact airport-airline summaries.

Data preparation affects both correctness and benchmark fairness. Correctness improves because each technology reads the same typed Parquet schema instead of reimplementing raw CSV parsing. Benchmark fairness improves because all technologies operate on the same generated inputs. The limitation is that preparation cost is not included in the per-job execution-time tables; it is a separate mandatory pipeline stage documented in this report.

Small input sizes can distort runtime comparisons. JVM startup, Spark session startup, Docker service overhead, Hive query startup, file listing, and query planning are fixed costs. At `100k`, those fixed costs can dominate elapsed time. This is why rows-per-second and normalized scalability ratios are included next to raw seconds: they show that larger inputs often process many more rows per second even when elapsed time is flat or non-monotonic.

9 Reproducibility And Validation

Generated large data, raw data, and full output directories are ignored by Git. The repository commits only small evaluator-facing evidence: report tables, figures, source code, documentation, and scripts.

Correctness validation is available through:

```
.\.venv\Scripts\python.exe scripts\validate_spark_sql_outputs.py
.\.venv\Scripts\python.exe scripts\validate_spark_core_outputs.py
.\.venv\Scripts\python.exe scripts\validate_hive_outputs.py
```

Spark Core and Hive validators compare their outputs against Spark SQL, including output columns, row counts, key sets, numeric values within tolerance, top-three cause labels and counts, ranking order, and first-10 sample files.

10 Limitations

- The raw dataset and generated benchmark inputs are not committed to Git because they are large.
- The raw dataset does not include airline names, so analyses use `airline_code`.
- MapReduce is out of scope.
- The Docker Spark setup is a standalone simulation on one physical machine, not true multi-machine cluster performance.
- Hive is containerized locally and is not running on a distributed Hadoop/YARN deployment.
- Worker-count variation was not used in M2 because the reliable Docker Compose topology has two named Spark workers.
- Windows Spark runs require care around Java, Hadoop `winutils.exe`, and native file handling; the project includes compatibility helpers and Docker paths where needed.
- Benchmark results are hardware-dependent and should be interpreted as evidence for this controlled environment, not universal performance numbers.

11 Conclusion

The project satisfies the main assignment requirements with a reproducible data preparation pipeline, two completed analyses, three technology implementations, first-10 result samples for each job and technology, benchmark tables, execution-time charts, and a critical discussion of expressiveness, implementation effort, efficiency, scalability, shuffle, aggregation, and data preparation effects. Spark SQL provides the clearest reference logic, Spark Core demonstrates lower-level distributed transformation control, and Hive provides the required SQL-on-Hadoop comparison point.

12 Evidence Appendix: First 10 Result Rows

The tables below embed the first 10 rows generated for both analytical jobs and all three selected technologies. Wide outputs are split into paired narrow tables so the PDF remains readable. The source artifacts are stored under `report/tables/` and are regenerated by `make charts`.

12.1 Spark SQL Samples

12.1.1 Delay Metrics

Origin	Mo.	Range	Flights	Avg dep.	Avg arr.
ABE	1	high	8	231.5	221.375
ABE	1	low	35	-6.286	-19.629
ABE	1	medium	4	32.75	31.25
ABE	2	high	3	324	335.667
ABE	2	low	40	-8.5	-23.575
ABE	2	medium	4	45.5	29.333
ABE	3	high	8	127	121.625
ABE	3	low	41	-4.756	-16.39
ABE	3	medium	3	32.333	12.667
ABE	4	high	6	158.333	150.333

12.1.2 Delay Top-Three Causes

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	1	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	1	low	unknown (34)	delay:nas (1)	none (0)
ABE	1	medium	delay:late_aircraft (2)	delay:carrier (1)	delay:nas (1)
ABE	2	high	delay:late_aircraft (1)	delay:nas (1)	delay:weather (1)
ABE	2	low	unknown (39)	delay:nas (1)	none (0)

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	2	medium	unknown (2)	delay:carrier (1)	delay:late_aircraft (1)
ABE	3	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	3	low	unknown (40)	delay:nas (1)	none (0)
ABE	3	medium	unknown (2)	delay:carrier (1)	none (0)
ABE	4	high	delay:carrier (4)	delay:late_aircraft (2)	none (0)

12.1.3 Ranking Metrics

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABE	9E	140	10.234	0.445	0.021
ABE	G4	260	12.463	4.337	0.012
ABE	OH	159	19.38	6.867	0.006
ABE	OO	40	25.297	14.389	0.075
ABI	MQ	249	6.664	4.405	0.008
ABQ	DL	218	1.383	-5.15	0.018
ABQ	OO	547	3.617	-0.855	0.004
ABQ	UA	273	4.584	-1.477	0.022
ABQ	MQ	127	7.134	5.575	0
ABQ	AS	67	7.522	1.881	0

12.1.4 Ranking Comparison

Origin	Airline	Airport avg dep.	Diff vs airport	Rank
ABE	9E	14.606	-4.373	1
ABE	G4	14.606	-2.143	2
ABE	OH	14.606	4.774	3
ABE	OO	14.606	10.691	4
ABI	MQ	6.664	0	1
ABQ	DL	8.792	-7.409	1
ABQ	OO	8.792	-5.176	2
ABQ	UA	8.792	-4.208	3
ABQ	MQ	8.792	-1.659	4
ABQ	AS	8.792	-1.27	5

12.2 Spark Core Samples

12.2.1 Delay Metrics

Origin	Mo.	Range	Flights	Avg dep.	Avg arr.
ABE	1	high	8	231.5	221.375
ABE	1	low	35	-6.286	-19.629
ABE	1	medium	4	32.75	31.25
ABE	2	high	3	324	335.667
ABE	2	low	40	-8.5	-23.575
ABE	2	medium	4	45.5	29.333
ABE	3	high	8	127	121.625
ABE	3	low	41	-4.756	-16.39
ABE	3	medium	3	32.333	12.667
ABE	4	high	6	158.333	150.333

12.2.2 Delay Top-Three Causes

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	1	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	1	low	unknown (34)	delay:nas (1)	none (0)
ABE	1	medium	delay:late_aircraft (2)	delay:carrier (1)	delay:nas (1)
ABE	2	high	delay:late_aircraft (1)	delay:nas (1)	delay:weather (1)
ABE	2	low	unknown (39)	delay:nas (1)	none (0)
ABE	2	medium	unknown (2)	delay:carrier (1)	delay:late_aircraft (1)
ABE	3	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	3	low	unknown (40)	delay:nas (1)	none (0)
ABE	3	medium	unknown (2)	delay:carrier (1)	none (0)
ABE	4	high	delay:carrier (4)	delay:late_aircraft (2)	none (0)

12.2.3 Ranking Metrics

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABE	9E	140	10.234	0.445	0.021
ABE	G4	260	12.463	4.337	0.012

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABE	OH	159	19.38	6.867	0.006
ABE	OO	40	25.297	14.389	0.075
ABI	MQ	249	6.664	4.405	0.008
ABQ	DL	218	1.383	-5.15	0.018
ABQ	OO	547	3.617	-0.855	0.004
ABQ	UA	273	4.584	-1.477	0.022
ABQ	MQ	127	7.134	5.575	0
ABQ	AS	67	7.522	1.881	0

12.2.4 Ranking Comparison

Origin	Airline	Airport avg dep.	Diff vs airport	Rank
ABE	9E	14.606	-4.373	1
ABE	G4	14.606	-2.143	2
ABE	OH	14.606	4.774	3
ABE	OO	14.606	10.691	4
ABI	MQ	6.664	0	1
ABQ	DL	8.792	-7.409	1
ABQ	OO	8.792	-5.176	2
ABQ	UA	8.792	-4.208	3
ABQ	MQ	8.792	-1.659	4
ABQ	AS	8.792	-1.27	5

12.3 Hive Samples

12.3.1 Delay Metrics

Origin	Mo.	Range	Flights	Avg dep.	Avg arr.
ABE	1	high	8	231.5	221.375
ABE	1	low	35	-6.286	-19.629
ABE	1	medium	4	32.75	31.25
ABE	2	high	3	324	335.667
ABE	2	low	40	-8.5	-23.575
ABE	2	medium	4	45.5	29.333
ABE	3	high	8	127	121.625
ABE	3	low	41	-4.756	-16.39
ABE	3	medium	3	32.333	12.667
ABE	4	high	6	158.333	150.333

12.3.2 Delay Top-Three Causes

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	1	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	1	low	unknown (34)	delay:nas (1)	none (0)
ABE	1	medium	delay:late_aircraft (2)	delay:carrier (1)	delay:nas (1)
ABE	2	high	delay:late_aircraft (1)	delay:nas (1)	delay:weather (1)
ABE	2	low	unknown (39)	delay:nas (1)	none (0)

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	2	medium	unknown (2)	delay:carrier (1)	delay:late_aircraft (1)
ABE	3	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	3	low	unknown (40)	delay:nas (1)	none (0)
ABE	3	medium	unknown (2)	delay:carrier (1)	none (0)
ABE	4	high	delay:carrier (4)	delay:late_aircraft (2)	none (0)

12.3.3 Ranking Metrics

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABE	9E	140	10.234	0.445	0.021
ABE	G4	260	12.463	4.337	0.012
ABE	OH	159	19.38	6.867	0.006
ABE	OO	40	25.297	14.389	0.075
ABI	MQ	249	6.664	4.405	0.008
ABQ	DL	218	1.383	-5.15	0.018
ABQ	OO	547	3.617	-0.855	0.004
ABQ	UA	273	4.584	-1.477	0.022
ABQ	MQ	127	7.134	5.575	0
ABQ	AS	67	7.522	1.881	0

12.3.4 Ranking Comparison

Origin	Airline	Airport avg dep.	Diff vs airport	Rank
ABE	9E	14.606	-4.373	1
ABE	G4	14.606	-2.143	2
ABE	OH	14.606	4.774	3
ABE	OO	14.606	10.691	4
ABI	MQ	6.664	0	1
ABQ	DL	8.792	-7.409	1
ABQ	OO	8.792	-5.176	2
ABQ	UA	8.792	-4.208	3
ABQ	MQ	8.792	-1.659	4
ABQ	AS	8.792	-1.27	5